FocusBoth

Predicting H-1B Sponsorship for OPT Job Seekers with DOL Labor Condition Application Data

An applied research project in data engineering, machine learning and decision support

Jessica Anguluri
Individual Applied Research Project

BAN 693 Business Analytics Capstone Project
Instructor: Dr. Surendra Sarnikar
California State University, East Bay
Fall 2026

Table of Contents

Abstract 3
1. Introduction and Business Problem 4
  1.1 Business Problem 4
  1.2 Statement of Purpose 4
  1.3 Research Objectives 4
  1.4 Why It Matters 4
2. Related Work 5
  2.1 Analytics on H-1B disclosure data 5
  2.2 Sponsorship lookup tools 5
  2.3 Occupation coding from job titles 5
  2.4 Decision support and grounded AI 5
  2.5 Gap addressed 5
3. Data Description 6
  3.1 Data Sources 6
  3.2 LCA Disclosure Data 6
  3.3 Features and Characteristics 6
  3.4 Limitations 8
4. Models 9
  4.1 System Architecture 9
  4.2 ETL Pipeline 9
  4.3 Dimensional Model 10
  4.4 M1: Employer Sponsorship Signal 11
  4.5 M2: Job Title to Occupation (Text Classification) 11
  4.6 M3: Sponsored Wage Regression 11
  4.7 Application and Model Deployment 12
5. Results 12
  5.1 M1: Employer Sponsorship Signal 12
    5.1.1 Robustness: a second test quarter 13
    5.1.2 Calibration and seasonality 13
    5.1.3 What drives the prediction 14
  5.2 M2: Job Title to Occupation 15
  5.3 M3: Sponsored Wage 15
  5.4 Results in the Application 17
6. Conclusions and Impact 18
  6.1 Conclusions 18
  6.2 Business Impact 18
  6.3 Limitations and Future Work 18
7. Contributions 19
  7.1 Individual Contribution 19
  7.2 Use of Generative AI 19
  7.3 Reflection on the Role of Generative AI 20
8. References 20
9. Appendix: Code 21

Abstract

International students on F-1 Optional Practical Training (OPT) must find an employer willing to sponsor an H-1B visa before their work authorization ends, yet the tools they use treat jobs, visa rules and personal time as separate problems. This project builds FocusBoth, a cloud-hosted decision-support web application, and grounds its sponsorship signal in 539,975 H-1B Labor Condition Applications (LCAs) published by the U.S. Department of Labor for fiscal quarters FY2025Q4 to FY2026Q3. An extract-transform-load pipeline standardizes 60,252 employer names and loads a star schema. Three predictive models are evaluated on a held-out final quarter: an employer-level classifier that predicts whether an employer will file a new-hire H-1B LCA in the next quarter, a text classifier that maps job titles to Standard Occupational Classification (SOC) codes, and a wage regression. Each model is compared with simple baselines. The resulting scores replace placeholder numbers in the application, which pulls live openings from 15 public employer job boards, applies deterministic visa-date rules, and keeps weekly hours for rest and hobbies in the same view.

1. Introduction and Business Problem

1.1 Business Problem

More than 1.1 million international students were enrolled in the United States in academic year 2024/25, a count that includes students on Optional Practical Training, and OPT participation grew sharply that year [1]. OPT gives F-1 graduates 12 months of work authorization, extended by 24 months for STEM degrees, with a cap of 90 unemployment days on standard OPT and 150 days in total for STEM OPT [2]. To stay in the country after OPT, most graduates need an employer to file an H-1B petition, which begins with a Labor Condition Application (LCA) filed with the Department of Labor (DOL) [3].

The practical problem is a decision under a hard deadline. A student must decide which openings to spend limited applications and interview time on, knowing that a job at an employer that does not sponsor is, for their purposes, not a job. Public job boards do not show sponsorship history. Sponsorship lookup sites show historical counts but do not connect them to today's openings or to the student's own Employment Authorization Document (EAD) end date. The same pressure also crowds out rest and personal time during a long search.

1.2 Statement of Purpose

The purpose of this project is to give an OPT student one place to answer a single business question: which current openings are realistic given my visa dates, and does my week still leave room for life outside the search? The analytics contribution is a sponsorship signal built from public DOL data and validated against later, unseen filings, instead of a guess or a static count.

1.3 Research Objectives

O1. Build a reproducible ETL pipeline and dimensional model (star schema) for DOL LCA disclosure data.

O2. Predict whether an employer will file a new-hire H-1B LCA in the next quarter, and test the prediction on a quarter the model never saw.

O3. Map free-text job titles from live postings to SOC occupation codes so each opening can be linked to DOL occupation and wage statistics.

O4. Model sponsored wages from information a student actually knows (occupation, state, wage level).

O5. Deliver the scores inside a working, cloud-hosted web application in which deterministic rules, not a language model, decide visa feasibility.

1.4 Why It Matters

The decision has real costs. A student who spends weeks on employers with no sponsorship history can exhaust the unemployment allowance, and a generic chatbot may state immigration rules incorrectly. A transparent, data-grounded signal that can be checked against public filings lowers that risk, while the app's rules engine keeps visa feasibility auditable.

2. Related Work

2.1 Analytics on H-1B disclosure data

Hooshmand et al. explored about 2.8 million LCA records from the DOL Office of Foreign Labor Certification (OFLC) and trained k-nearest-neighbor, random forest and multilayer perceptron classifiers to predict whether a case would be certified, reporting an F1 score near 0.98 [4]. Ke and Qiao used LCA data from 2008–2018 with LASSO regression for wages and L1-penalized logistic regression for certification, finding that occupation, state and job level drive wages [5].

Both studies predict the outcome of a single application. That target is of little use to a job seeker, because over 90% of LCAs are certified (91.4% in this project's data) and the student does not file the LCA. This project instead predicts employer behavior over time: will this employer file new-hire H-1B LCAs next quarter? It also tests on a later quarter rather than a random split, which matches how the signal would be used in practice.

2.2 Sponsorship lookup tools

Commercial sites such as MyVisaJobs [6] publish employer-level LCA counts as a searchable directory. They are valuable references, but they report history without testing whether it predicts future filings. They also are not connected to live openings or to a student's visa dates.

2.3 Occupation coding from job titles

Assigning SOC codes to free-text titles is a known problem in labor economics and epidemiology. SOCcer scores job titles and task descriptions against SOC definitions for research coding [7], and Rony and Patman fine-tuned BERT models on hand-labelled UK job advertisements, reaching 61% accuracy at the most detailed SOC tier [8]. This project uses a lighter TF-IDF and logistic-regression model trained on about half a million employer-assigned title–SOC pairs from LCAs. The labels come for free with the public data and the model is small enough to run inside the web server.

2.4 Decision support and grounded AI

Retrieval-augmented generation restricts a language model to retrieved source passages [9], and gradient-boosted trees such as XGBoost [10] are strong baselines for tabular prediction. FocusBoth applies both ideas conservatively. Gradient boosting produces a probability that is checked for calibration [11], and any language-model text is limited to explaining outputs that the deterministic rules engine has already computed.

2.5 Gap addressed

No reviewed tool combines (a) a forward-looking sponsorship signal validated on unseen quarters, (b) live job openings linked to occupation and wage data, and (c) a student's own EAD dates and unemployment allowance, in one application. FocusBoth integrates these pieces. Its contribution is the integration and the validation, not a new statistical method.

3. Data Description

3.1 Data Sources

The project integrates five data sources (Table 1). The DOL LCA disclosure files are the primary analytical dataset; the others feed the application.

Table 1. *Data sources integrated in FocusBoth*

Source	Access	Refresh	Role in project
U.S. DOL OFLC LCA disclosure files [3]	Public quarterly Excel files	Quarterly (batch)	Primary analytics dataset; star schema; models M1–M3
Greenhouse Job Board API [15]	Public JSON, no key	30 minutes	Live openings from 15 employers; exact apply links
Devpost hackathon listings	Public JSON	30 minutes	Discover screen (hackathons)
confs.tech conference data [16]	Open GitHub JSON	30 minutes	Discover screen (tech conferences)
Student-entered profile	Typed into the app	Live	Visa dates, unemployment days, weekly wellbeing goals

3.2 LCA Disclosure Data

The U.S. Department of Labor, Office of Foreign Labor Certification, publishes quarterly disclosure files of every LCA it decides [3]. Each row is one application, with fields for the employer, job title, SOC code, worksite, offered wage, prevailing wage and level, case status and the number of workers by employment type. Files are partitioned by decision date. This project uses the FY2025 Q4 file (decisions July–September 2025) and the cumulative FY2026 Q3 file (decisions October 2025–June 2026), giving four consecutive fiscal quarters. The FY2026 Q1 file was also downloaded, but all of its 83,120 cases are contained in the cumulative Q3 file, so it was excluded to avoid double counting.

3.3 Features and Characteristics

After cleaning, the dataset contains 539,975 H-1B LCAs from 60,252 distinct employers across 497 SOC occupations and 55 worksite states and territories. Activity is highly concentrated. The ten largest filers account for 13.8% of all filings, while 28,231 employers filed exactly once. Computer and mathematical occupations (SOC major group 15) make up 60.6% of filings. 49.3% of filings include at least one new-employment or change-of-employer position, the type an OPT graduate would need. The median offered wage on certified cases is $126,256.

The data are strongly seasonal. FY2026 Q3 (April–June 2026) has 228,103 filings, about twice the other quarters, and 64.7% of them involve new hires, compared with roughly 37–40% in the other quarters. This reflects the annual H-1B cap cycle and matters for model evaluation (Section 5.1).

Figure 1 shows the seasonal pattern. Figures 2 to 4 show where filings concentrate: by employer, Amazon.com Services LLC (16,502), Google LLC (8,280), COGNIZANT TECHNOLOGY SOLUTIONS US CORP (8,277); by occupation, Software Developers (162,907), Computer Systems Engineers/Architects (41,794), Data Scientists (34,622); and by worksite state, CA (103,204), TX (81,363), NY (45,212).

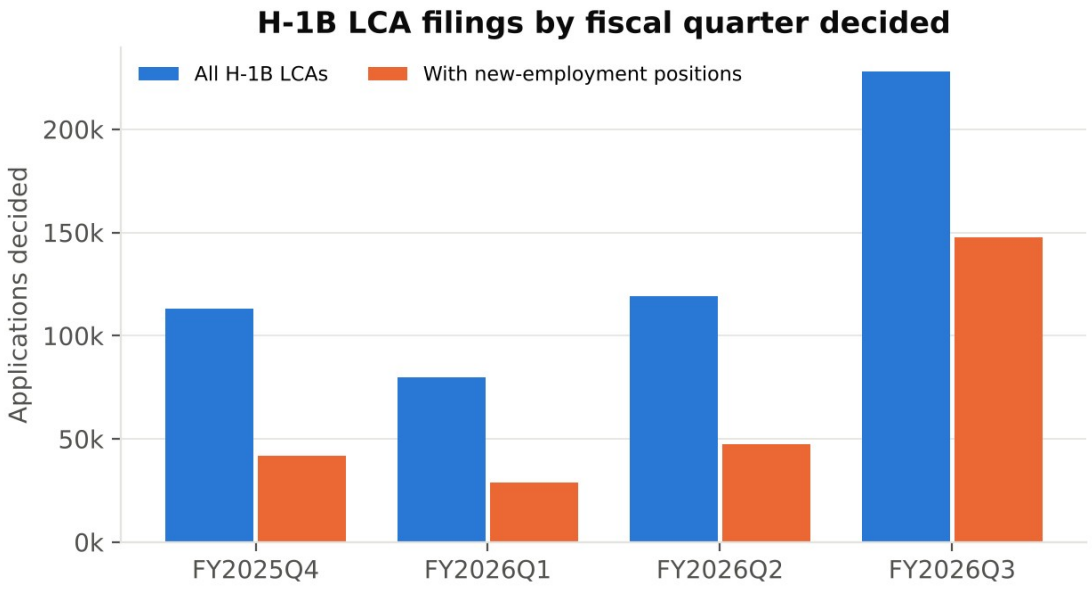


Figure 1. *H-1B LCAs by fiscal quarter of decision, all filings and filings with new-employment positions*

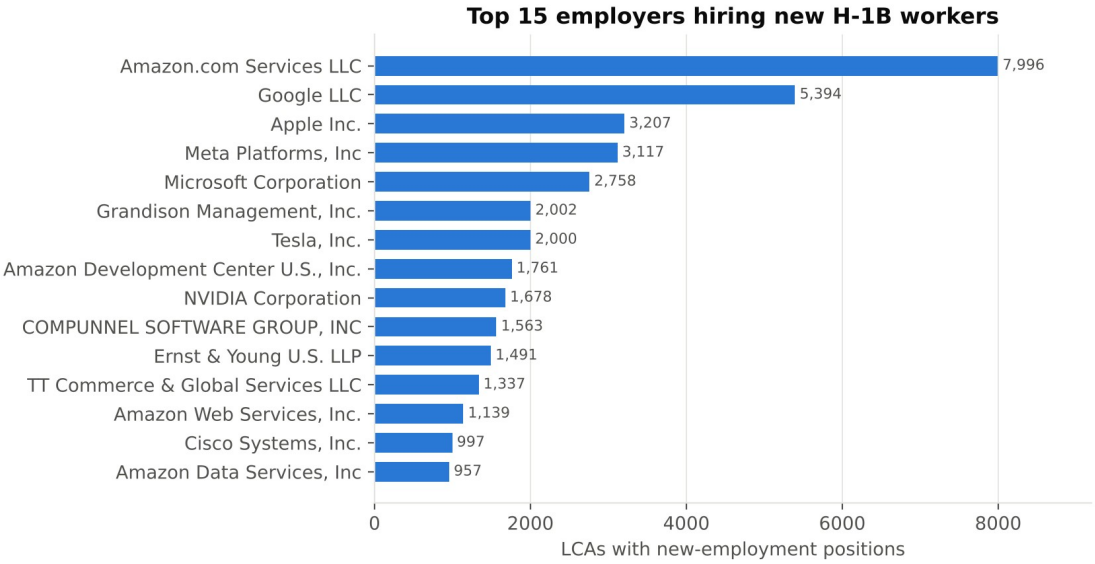


Figure 2. *Top 15 employers by LCAs with new-employment positions*

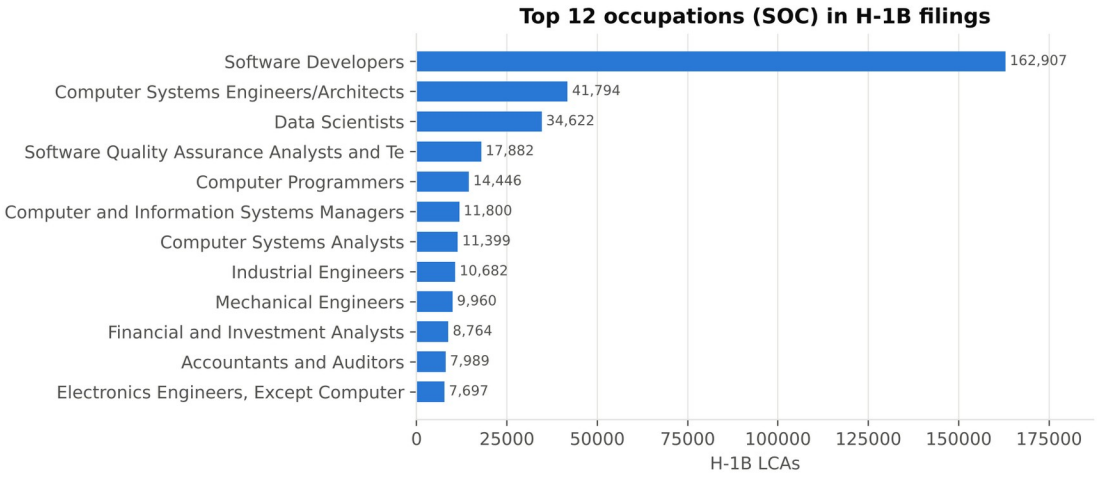


Figure 3. *Top 12 SOC occupations in H-1B LCAs*

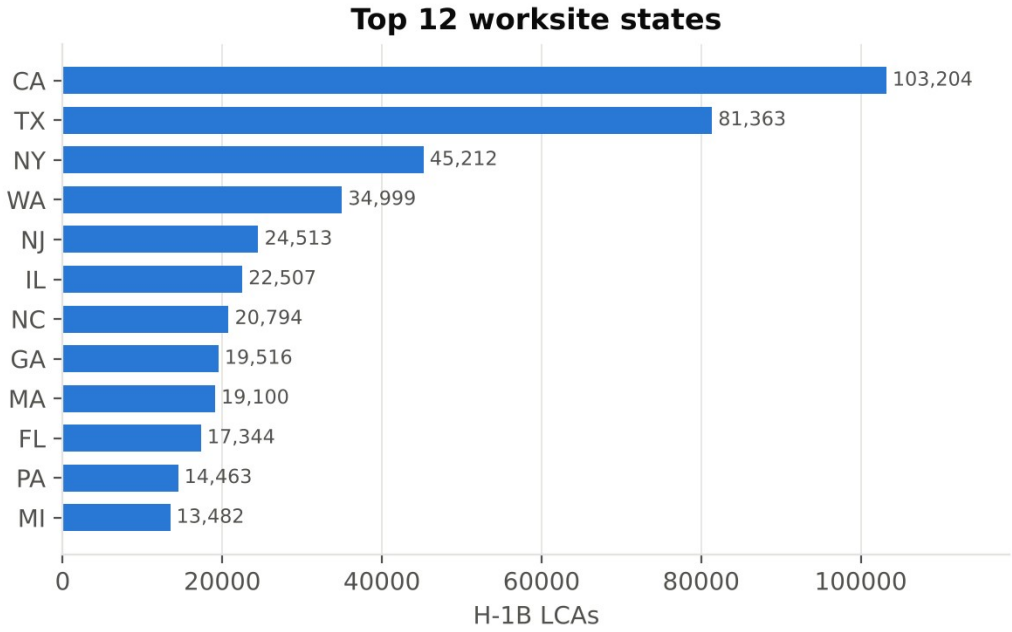


Figure 4. *Top 12 worksite states by H-1B LCAs*

3.4 Limitations

- An LCA is a precondition for an H-1B petition, not a petition or an approval. A filed LCA does not mean a worker was hired or that a visa was granted.
- Employer names are self-reported. Standardization merges common spelling variants, but subsidiaries and staffing agencies may still be split or combined incorrectly. Some brands file under a different legal name (Instacart as Maplebear, Airtable as Formagrid), which the application handles with an explicit mapping.
- SOC codes are assigned by employers, so the title-to-SOC labels contain noise, and similar roles may be coded differently by different employers.

- Four quarters is a short history. The test quarter falls in the cap season, so its new-hire rate differs from the training quarters.
- Live job coverage is limited to the 15 employers whose public Greenhouse boards are configured, not the whole U.S. labor market.

4. Models

4.1 System Architecture

Figure 5 shows the end-to-end pipeline. Raw disclosure workbooks are extracted with only the 32 needed columns (the full workbook does not fit in memory), transformed in pandas, and loaded into a DuckDB warehouse and Parquet files. The same star schema is created in a managed cloud PostgreSQL database, from which the web application reads. Models are trained in Python with scikit-learn [12] and XGBoost [10], and their outputs are exported as compact JSON files consumed by the Next.js server.

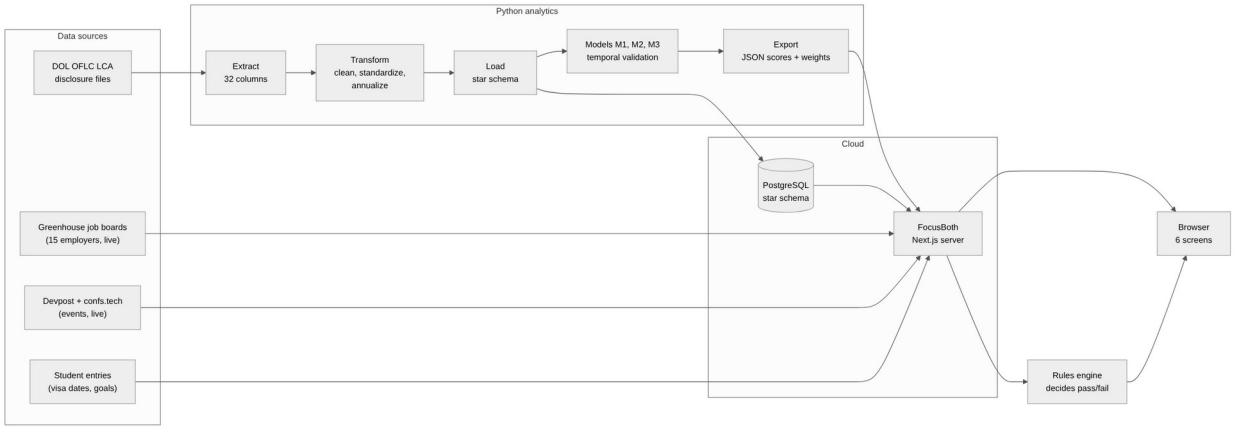


Figure 5. *End-to-end architecture: ETL, warehouse, models and the FocusBoth application*

4.2 ETL Pipeline

Table 2. *ETL steps implemented in analytics/etl.py*

Step	What it does
Extract	Read each quarterly workbook, keep the needed columns, and tag rows with their source file.
Deduplicate	Keep one row per CASE_NUMBER; 1,886 duplicate case numbers across files were removed, keeping the most recent decision.
Filter	Keep VISA_CLASS = H-1B (14,214 H-1B1 and E-3 rows removed).
Standardize employers	Upper-case, remove punctuation, drop ‘doing business as’ clauses and legal suffixes (Inc., LLC, Corp., PBC, …). The same rule is implemented in Python and TypeScript and tested for identical output.
Annualize wages	Convert hourly, weekly, bi-weekly and monthly wages to annual (hour × 2,080, week × 52, bi-week × 26, month × 12). 830 implausible wages (below $20,000 or above $1,000,000 per year) were set to missing rather than dropped, so the filing still counts.
Derive fields	Fiscal quarter from decision date (federal fiscal year starts 1 October), SOC code to the six-digit XX-XXXX form, state names to two-letter codes, new-hire positions = new employment + change of employer.

Step	What it does
Load	Build dimension tables with surrogate keys and the fact table, then write to DuckDB, Parquet and PostgreSQL.

4.3 Dimensional Model

The dimensional model (Figure 6) follows Kimball's star-schema design [13]. The grain of fact_lca is one LCA case. Four conformed dimensions describe who filed (dim_employer), for what occupation (dim_occupation), where (dim_location) and when (dim_date, keyed by decision date with fiscal-quarter attributes). Measures include total, new-hire and continued positions and the annualized offered and prevailing wages. Flags record certified, withdrawn and denied status. This design lets every BI figure in Section 5 be produced by a short join-and-group query.

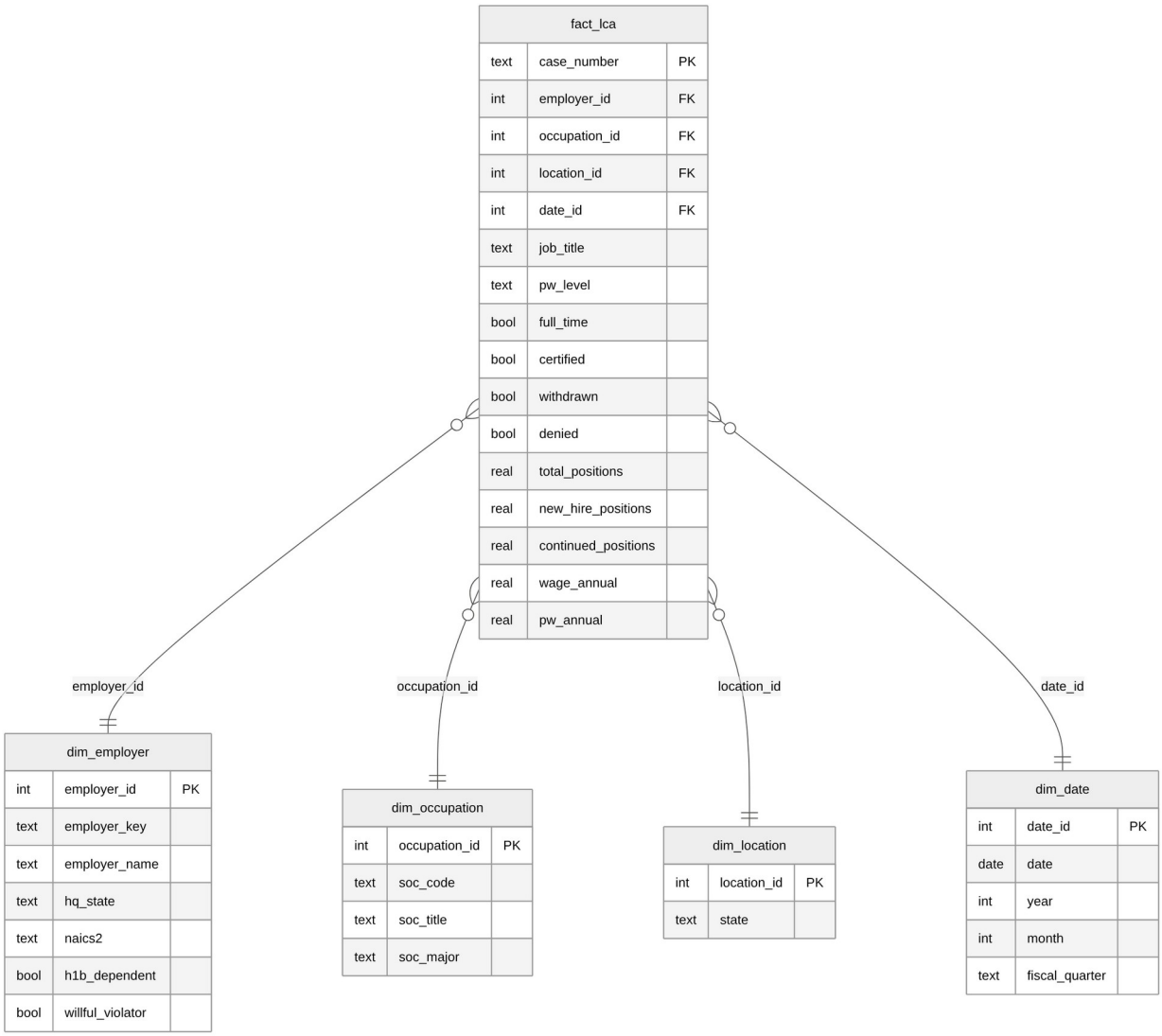


Figure 6. *Star schema: fact_lca with employer, occupation, location and date dimensions*

4.4 M1: Employer Sponsorship Signal

Target. For each employer that had filed at least once by the end of quarter t, the label is 1 if it files at least one LCA with new-employment or change-of-employer positions in quarter t + 1, and 0 otherwise. This is the event that matters to an OPT graduate looking for a new sponsor.

Features. Features use only information available at the end of quarter t, so there is no look-ahead. They cover historical and current-quarter filing counts, new-hire counts, quarters active, quarters since the last filing, certification and withdrawal rates, occupational and geographic breadth, share of computer occupations and of Level I wages, median wage and wage-to-prevailing-wage ratio, H-1B-dependent status and two-digit industry (NAICS). Even the employer attributes are computed from quarters up to t only.

Models. Two baselines: B0 persistence (did the employer file a new-hire LCA this quarter?) and B1 frequency (count of historical new-hire filings). Two learned models: L2-regularized logistic regression with class weighting on standardized features, and XGBoost (400 trees, depth 5, learning rate 0.05).

Validation. A temporal split. Models train on employer-quarters whose label quarter is FY2026 Q1 or Q2 and are tested on label quarter FY2026 Q3. Metrics are ROC-AUC, PR-AUC (more informative when positives are a minority [14]), precision among the top 1% of ranked employers, and the Brier score for probability quality [11].

4.5 M2: Job Title to Occupation (Text Classification)

Target. The employer-assigned six-digit SOC code, restricted to the 60 most frequent codes, which cover 90.2% of filings.

Models. A majority-class baseline; an exact-title lookup baseline (the most common SOC for an identical cleaned title in training); and TF-IDF over word unigrams and bigrams (sublinear term frequency, up to 20,000 features) feeding a multinomial logistic regression.

Validation. A temporal split, testing on FY2026 Q3. Because a lookup table trivially succeeds on titles it has already seen, accuracy is also reported separately for test titles that never appeared in training, which is the case that matters for new job postings.

4.6 M3: Sponsored Wage Regression

Target. Log annual offered wage on certified LCAs.

Features. Features are restricted to what a student can know before applying: SOC occupation, worksite state, prevailing-wage level (I–IV, a proxy for experience), full-time status and industry. The prevailing-wage amount itself is excluded because it nearly determines the offered wage and a student would not know it.

Models. A baseline (median wage by SOC), ridge regression on one-hot features, and XGBoost (600 trees, depth 7). The split is temporal, with FY2026 Q3 held out. Metrics are mean absolute error in dollars, mean absolute percentage error, and R^2 on the log scale.

4.7 Application and Model Deployment

FocusBoth is a Next.js 16 / React 19 / TypeScript application with six screens: Today, Visa Clock, Job Hunt, Path Navigator, Wellbeing and Discover. A deterministic rules engine (lib/compass-engine.ts) decides feasibility. A job passes only if the sponsor signal is at least 60, the student's own résumé-fit rating is at least 65, and the start date falls on or before the EAD end date. A career path passes only if it starts before the EAD end, involves at least 20 hours a week, uses an E-Verify employer and has a bona fide employer relationship.

The models connect to the application in three places. (1) The live-openings API attaches each employer's DOL record and model M1 probability to every posting; the sponsor signal shown and used by the rules engine is that probability × 100. (2) Each job title is classified by model M2, which is re-implemented in TypeScript from exported weights and tested for identical predictions against Python, and the predicted occupation is shown with the DOL entry-level (Level I) median H-1B wage for it. (3) A lookup endpoint returns the DOL record for any employer a student types, using the same name standardization as the ETL. If an employer has no filings, the app shows zero and says so. It never fills in a guess.

5. Results

5.1 M1: Employer Sponsorship Signal

The employer–quarter panel has 88,391 rows: 50,915 for training (label quarters FY2026 Q1–Q2) and 37,476 for testing (label quarter FY2026 Q3). The share of employers filing a new-hire LCA in the following quarter rises from 23.3% in training to 38.3% in the test quarter, because the test quarter is the H-1B cap season.

Table 3. *M1 results on the held-out quarter FY2026 Q3 (bootstrap 95% confidence intervals in brackets, 1,000 resamples)*

Model	ROC-AUC	PR-AUC	Precision, top 1% (k=375)	Brier
B0 Persistence (baseline)	0.633 [0.628, 0.638]	0.482 [0.475, 0.489]	62.4%	–
B1 Frequency (baseline)	0.702 [0.697, 0.708]	0.622 [0.615, 0.630]	96.3%	–
Logistic regression	0.770 [0.765, 0.775]	0.724 [0.718, 0.732]	98.4%	0.184
XGBoost	**0.773** [0.768, 0.778]	**0.728** [0.721, 0.736]	99.5%	0.202

Both learned models clearly beat the baselines. XGBoost reaches ROC-AUC 0.773 and PR-AUC 0.728, against 0.702 and 0.622 for the best baseline, a count of past new-hire filings. The bootstrap 95% interval for the AUC gain of XGBoost over that baseline is [0.067, 0.075], which excludes zero. XGBoost and logistic regression perform almost identically, and their intervals overlap. Among the 375 employers ranked highest by XGBoost, 99.5% did file new-hire LCAs in the test quarter.

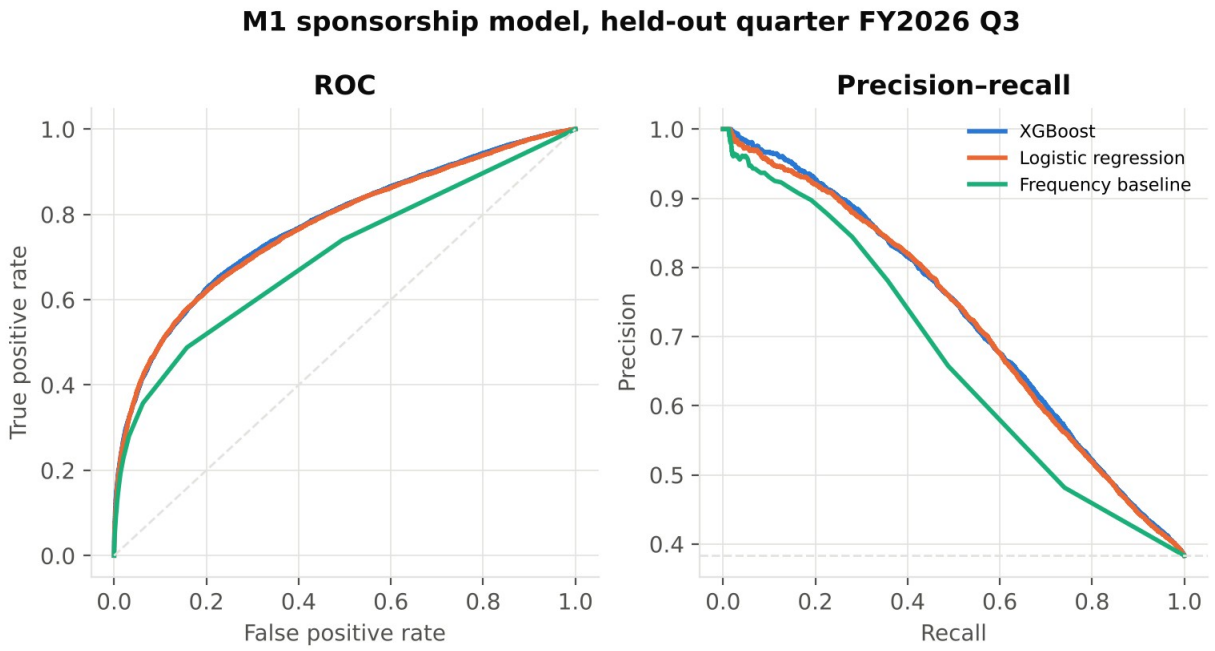


Figure 7. *ROC and precision–recall curves for M1 on the held-out quarter; dashed lines show chance and the positive rate*

5.1.1 Robustness: a second test quarter

To check that the result does not depend on one quarter, the models were retrained on label quarter FY2026 Q1 alone and tested on FY2026 Q2, a non-cap-season quarter with a 23.0% positive rate. Both learned models again beat both baselines, with ROC-AUC 0.810 for logistic regression and 0.798 for XGBoost, against 0.765 for the frequency baseline and 0.682 for persistence. PR-AUC values are 0.648, 0.641, 0.558 and 0.353 respectively. On this fold logistic regression is slightly ahead, so the two learned models should be regarded as tied. The robust conclusion is that a multi-feature model beats any single historical count, not that one learner is superior. XGBoost is used in the application because it is best on the larger, more recent test quarter and needs no feature scaling.

5.1.2 Calibration and seasonality

Ranking is good, but the probabilities need care. Figure 8 shows that, on the cap-season test quarter, XGBoost under-predicts (Brier 0.202) because it learned a 23.3% base rate, while class-weighted logistic regression over-predicts (Brier 0.184). For FocusBoth, the score is therefore best read as a ranking, meaning “this employer files new-hire LCAs more reliably than most”, rather than an exact probability. With more years of data, a quarter-of-year feature or per-season recalibration would address this.

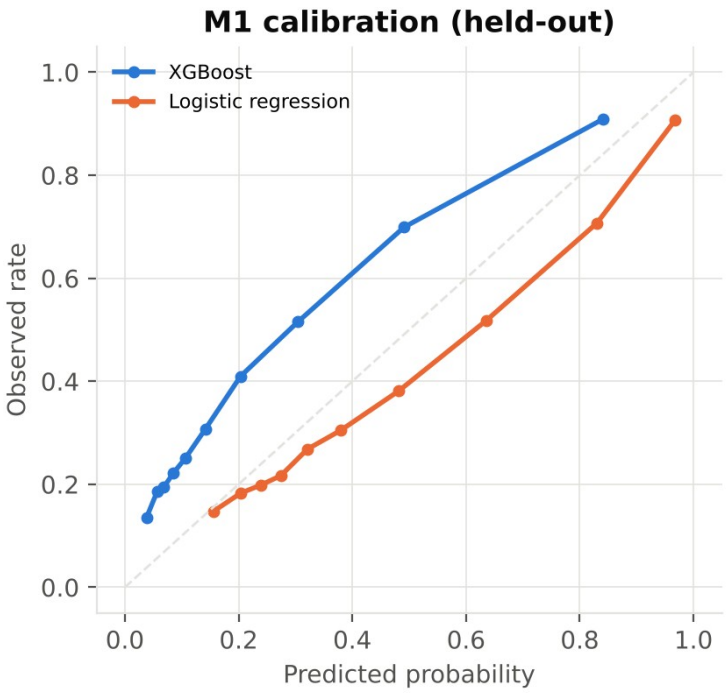


Figure 8. *M1 reliability diagram on the held-out quarter (10 quantile bins)*

5.1.3 What drives the prediction

The most important XGBoost features (Figure 9) are total past filings, past new-hire filings, number of distinct occupations filed, quarters with any filing, filings in the current quarter. Logistic-regression coefficients agree in sign. Employers with more historical new-hire filings, a broader range of occupations, more active quarters and H-1B-dependent status are more likely to file again. In plain terms, sponsorship is a habit. Employers that have built an immigration process keep using it, which is exactly the signal a student cannot see on a job board.

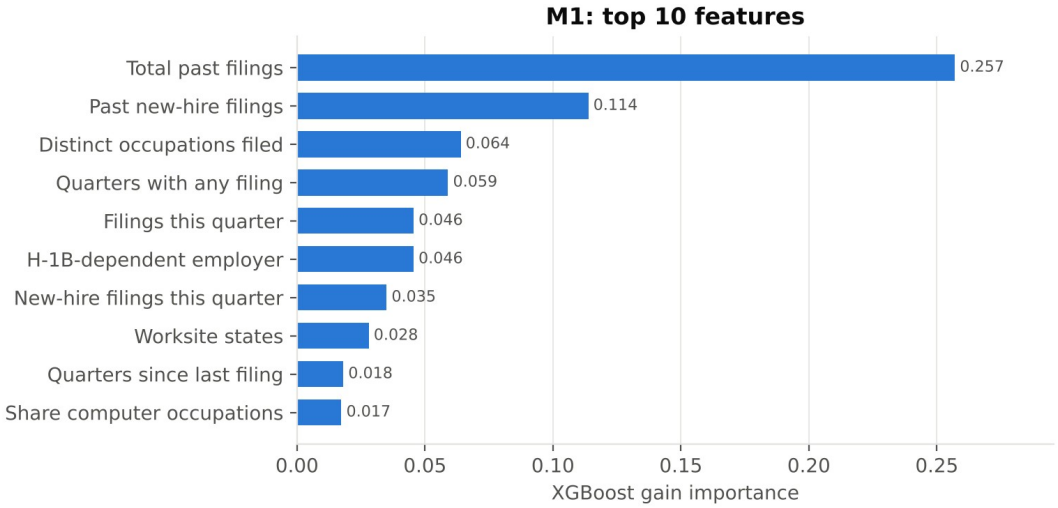


Figure 9. *Top 10 M1 XGBoost features by gain*

5.2 M2: Job Title to Occupation

Table 4. *M2 results on the held-out quarter (60 SOC classes; 203,559 test titles)*

Model	Top-1 accuracy	Top-3 accuracy	Macro F1	Accuracy, unseen titles
Majority class (baseline)	30.4%	–	0.008	–
Exact-title lookup (baseline)	63.0%	–	0.534	15.0%
TF-IDF + logistic regression	**70.4%**	87.0%	0.607	**51.8%**

The text model assigns the employer's own SOC code to 70.4% of test titles and includes it in its top three 87.0% of the time (95% CI for top-1 ±0.2 percentage points). The more telling comparison is on the 20.1% of test titles never seen in training, the situation for a new job posting. There, the lookup table can only fall back to the majority class (15.0%), while the model still reaches 51.8%. Some remaining error is label noise, because employers code near-identical roles (for example, software engineer) to different SOC codes. The compact version deployed in the application (3,000 features) scores 69.4% on the same held-out quarter.

5.3 M3: Sponsored Wage

Table 5. *M3 results on the held-out quarter (212,634 certified LCAs)*

Model	MAE	MAPE	R^2 (log wage)
Median wage by SOC (baseline)	$33,798	26.4%	0.356
Ridge regression	$22,453	16.2%	0.687
XGBoost	**$20,632**	**14.8%**	**0.730**

XGBoost reduces mean absolute error from $33,798 to $20,632 (bootstrap 95% CI $20,501–$20,756) and explains 73.0% of the variance in log wage. By total gain, occupation dominates (88.0%), followed by worksite state and wage level. Wage level matters a great deal within an occupation: for Software Developers the median offered wage is $108,000 at Level I and $178,602 at Level IV (Figure 10). Level I is the entry level at which most OPT graduates are hired, which is why FocusBoth displays the Level I median for each predicted occupation.

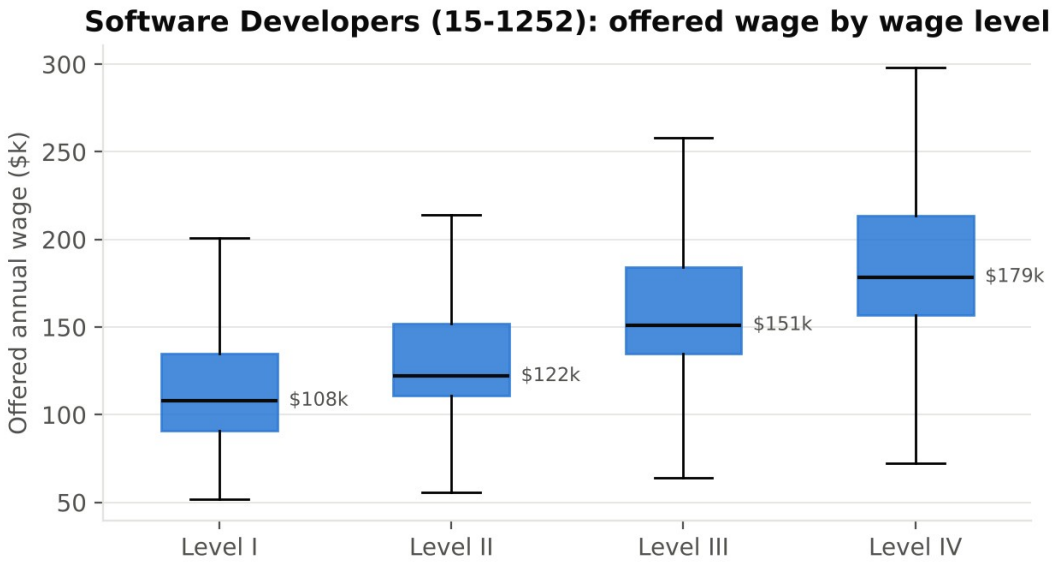


Figure 10. *Offered annual wage by prevailing-wage level, Software Developers (SOC 15-1252), certified LCAs*

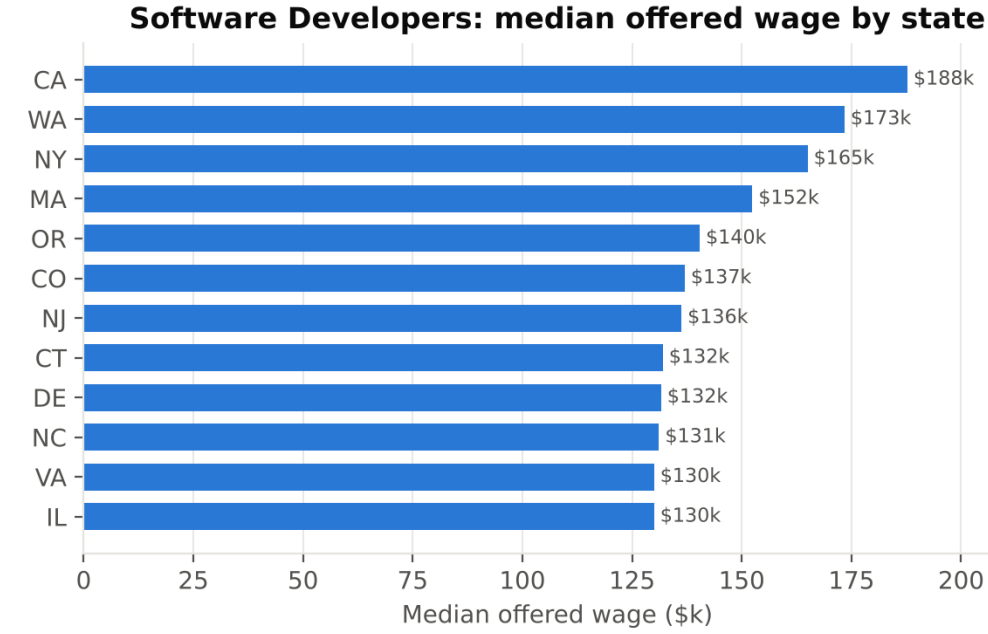


Figure 11. *Median offered wage for Software Developers by worksite state (states with at least 200 LCAs)*

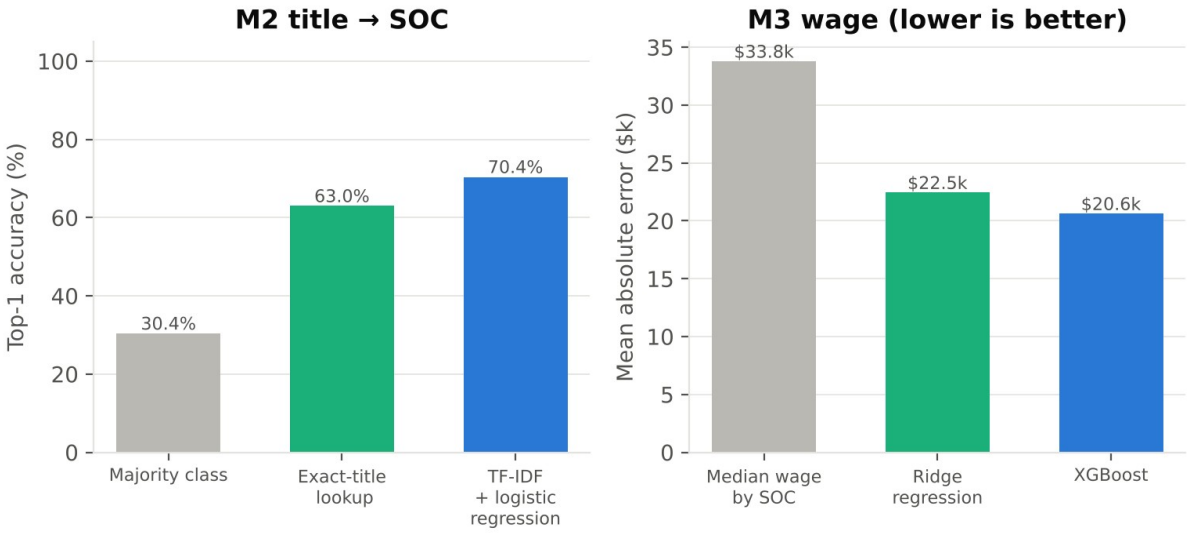


Figure 12. *Model versus baseline: M2 top-1 accuracy (left) and M3 mean absolute error (right) on the held-out quarter*

5.4 Results in the Application

Figure 13 shows the M1 signal for the 15 employers in FocusBoth's live feed. Fourteen have hundreds of filings and scores of 98–100. Two brands appear only under their legal names (Instacart as Maplebear and Airtable as Formagrid), and GitLab has no H-1B filings at all in the period. Under the rules engine, a student tracking a GitLab opening would therefore see the job fail the sponsorship rule, information that is not on GitLab's job board. The exported lookup table covers 20,841 employers with at least three filings, and any of them can be checked by name in the application.

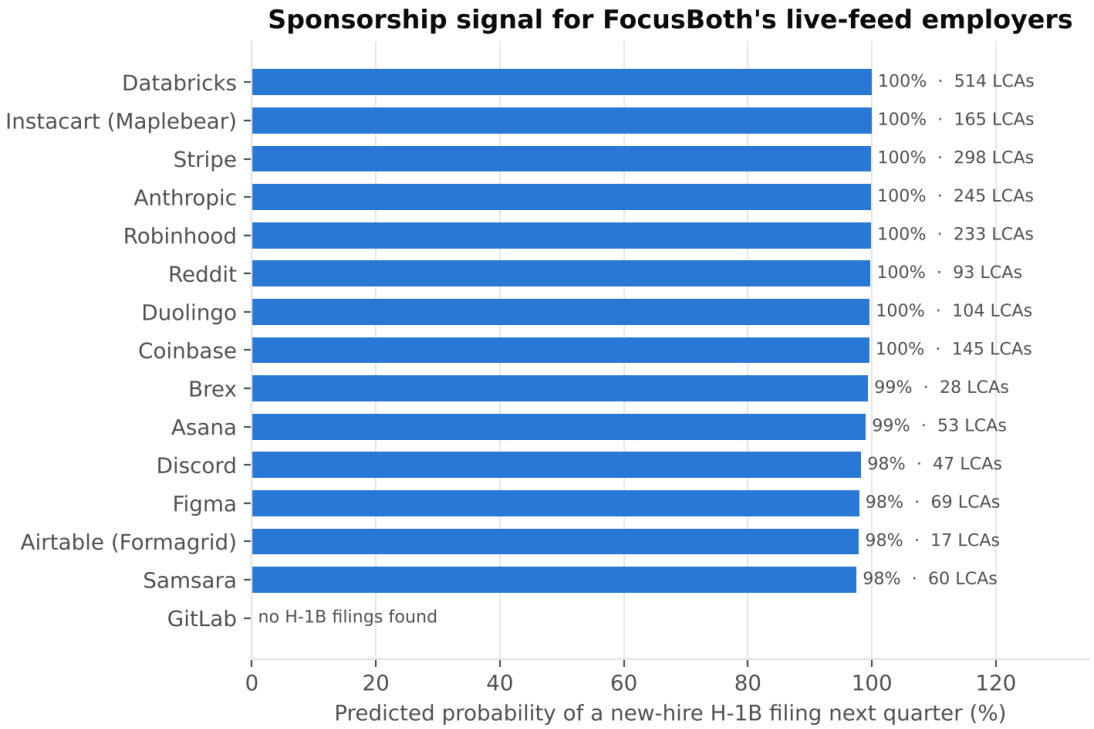


Figure 13. *M1 sponsorship signal and LCA count for the 15 employers in FocusBoth's live job feed*

Table 6. *Verification checks on the deployed system*

Check	Result
Python unit tests (ETL normalization, wages, fiscal quarters)	7 / 7 pass
TypeScript unit tests (rules engine, name normalization parity, lookup, title-model parity)	24 / 24 pass
TypeScript title model vs Python on 200 held-out titles	200 / 200 identical predictions
Lint, type check and production build	Pass (ESLint clean, tsc clean, next build succeeds)

6. Conclusions and Impact

6.1 Conclusions

This project shows that public DOL disclosure data can be turned into a forward-looking, validated sponsorship signal and delivered where a job seeker needs it. On a quarter none of the models saw, the employer model ranks future new-hire sponsors with ROC-AUC 0.773, a statistically significant gain over simply counting past filings, and the gain holds on a second test quarter. The title model links 70.4% of job titles to the exact occupation code, and more than half of titles it has never seen. The wage model cuts error by 39.0% relative to occupation medians.

Three findings matter for students directly. First, sponsorship is concentrated and persistent. A small number of employers file most LCAs, and past behavior is the strongest predictor of future behavior. Second, the market is seasonal. April–June carries about twice the usual volume and most new-hire filings, so an OPT graduate's search should be planned around the cap cycle. Third, brand names hide legal entities, and some well-known tech employers file no H-1B LCAs at all. A student can only learn this from the filings, not from a careers page.

6.2 Business Impact

- **For OPT students:** fewer applications spent on employers that have no sponsorship history, which protects the limited unemployment allowance (90 days, or 150 for STEM OPT).
- **For university career and international-student offices:** a transparent, reproducible way to advise students, built entirely on public data, with every number traceable to a DOL case record.
- **For the analytics method:** the project shows a pattern of letting deterministic rules decide and letting data and models inform. Feasibility stays auditable even though machine-learning scores are inputs.

6.3 Limitations and Future Work

- Extend the history to several fiscal years so seasonality can be modelled explicitly and probabilities recalibrated by season.
- Link LCAs to USCIS H-1B petition and approval data, which would let the system distinguish an LCA filed from a worker actually hired.

- Add entity resolution for subsidiaries and staffing firms beyond exact normalized-name matching.
- Expand live jobs beyond 15 Greenhouse boards (for example Lever boards or USAJOBS) without scraping sites whose terms prohibit it.
- Run a usability study with international students to measure whether the signal changes which jobs they pursue.

FocusBoth is informational decision support. It does not provide legal advice or determine immigration status, and every visa-related screen directs the student to confirm with their Designated School Official.

7. Contributions

7.1 Individual Contribution

This is an individual project. Jessica Anguluri defined the business problem and scope, selected the data sources and obtained the DOL disclosure files, and designed the product (the six screens and the rule that the rules engine, not a model, decides feasibility). She made the modelling decisions reviewed in this report (the new-hire sponsorship target, the temporal validation, and excluding the prevailing wage from M3), verified results against the source files, and wrote and presented the final work.

7.2 Use of Generative AI

The course requires generative AI to be used in every phase and credited. Table 7 lists where it was used. All AI output was reviewed, run and checked by the author, and every number in this report is produced by code in the appendix rather than typed by hand.

Table 7. *Generative AI tools and how they were used*

Phase	Tool	What the AI did	How it was verified
Prototype web app	Cursor (AI code editor)	Scaffolded the Next.js screens, API routes and design system	Manual testing of each screen; lint and build
Data engineering	Claude (Anthropic)	Drafted the ETL, star-schema load and export code	Unit tests; row counts reconciled against source files
Modelling	Claude (Anthropic)	Drafted model and robustness code; suggested the temporal split and the unseen-title test	Re-run end to end; leakage reviewed; baselines compared
App integration	Claude (Anthropic)	Wrote the TypeScript title model, lookup API and tests	Parity test against Python; vitest suite
Writing	Claude (Anthropic)	Drafted report sections and figure captions from the computed results	Edited by the author; numbers generated from metrics files; references checked against sources

7.3 Reflection on the Role of Generative AI

Generative AI made a two-day build of a full pipeline possible, but it did not replace judgement. Several of the most important decisions came from checking AI-written code against the data. Assigning quarters by decision date rather than received date matched how DOL partitions its files. Removing a look-ahead leak in employer attributes kept the test honest. Merging 'd/b/a' names correctly attributed Airtable's filings. Reporting accuracy on unseen titles prevented a lookup table from looking as good as a real model. The working rule throughout was that AI drafts, the data decides, and the author verifies.

8. References

[1] BUILA, "US tops 1.1 million international students as OPT fuels growth," British Universities' International Liaison Association, Nov. 2025, reporting Institute of International Education Open Doors 2024/25 data. [Online]. Available: https://www.buila.ac.uk/articles/us-tops-1-1-million-international-students-as-opt-fuels-growth

[2] U.S. Citizenship and Immigration Services, "Optional Practical Training (OPT) for F-1 Students." [Online]. Available: https://www.uscis.gov/working-in-the-united-states/students-and-exchange-visitors/optional-practical-training-opt-for-f-1-students

[3] U.S. Department of Labor, Employment and Training Administration, Office of Foreign Labor Certification, "OFLC Performance Data: LCA Programs (H-1B, H-1B1, E-3) Disclosure Data, FY2025 Q4 and FY2026 Q3." [Online]. Available: https://www.dol.gov/agencies/eta/foreign-labor/performance

[4] H. Hooshmand, J. Martinsen, J. Arauco, A. Dholasaniya, and B. Bhatt, "An exploration of H-1B visa applications in the United States," arXiv:1806.03368, 2018.

[5] B. S. Ke and A. Qiao, "Who gets the job and how are they paid? Machine learning application on H-1B case data," arXiv:1904.10580, 2019.

[6] MyVisaJobs, "H-1B Visa Sponsors and Employers." [Online]. Available: https://www.myvisajobs.com

[7] D. E. Russ et al., "Computer-based coding of free-text job descriptions to efficiently identify occupations in epidemiological studies," Occupational and Environmental Medicine, vol. 73, no. 6, pp. 417–424, 2016.

[8] S. Rony and J. Patman, "Standard occupation classifier: A natural language processing approach," arXiv:2511.23057, 2025.

[9] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Advances in Neural Information Processing Systems, vol. 33, 2020, pp. 9459–9474.

[10] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2016, pp. 785–794.

[11] A. Niculescu-Mizil and R. Caruana, "Predicting good probabilities with supervised learning," in Proc. 22nd Int. Conf. Machine Learning (ICML), 2005, pp. 625–632.

[12] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[13] R. Kimball and M. Ross, The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling, 3rd ed. Indianapolis, IN, USA: Wiley, 2013.

[14] T. Saito and M. Rehmsmeier, “The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets,” PLoS ONE, vol. 10, no. 3, e0118432, 2015.

[15] Greenhouse Software, “Job Board API.” [Online]. Available: https://developers.greenhouse.io/job-board.html

[16] Tech Conferences, “conference-data,” GitHub repository. [Online]. Available: https://github.com/tech-conferences/conference-data

9. Appendix: Code

The complete source code, including the Next.js application, the charts and export scripts, and all tests, is in the public repository (GitHub link added at publication). To reproduce the analysis, place the DOL disclosure files in data/raw/ and run, in order: etl.py, models.py, robustness.py, export.py, export_title_model.py, charts.py and report_facts.py. The key files are listed below.

`analytics/etl.py` *(ETL and star-schema load)*

```
"""
FocusBoth analytics — ETL for U.S. DOL OFLC LCA disclosure data.

Extract : read quarterly LCA disclosure workbooks (xlsx/csv) from data/raw/
Transform: keep H-1B rows, standardise employer names, annualise wages,
           derive fiscal quarter, clean SOC codes, flag new-employment filings
Load    : write a star schema into DuckDB (data/warehouse.duckdb) and parquet
          (data/star/*.parquet) so the same tables can be pushed to cloud Postgres.

Run:  python analytics/etl.py
"""
from __future__ import annotations

import glob
import os
import re
import sys

import duckdb
import numpy as np
import pandas as pd

ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
RAW = os.path.join(ROOT, "data", "raw")
STAR = os.path.join(ROOT, "data", "star")
DB = os.path.join(ROOT, "data", "warehouse.duckdb")

# Columns used from the DOL record layout (FY2020+). Missing columns are tolerated.
COLS = [
    "CASE_NUMBER", "CASE_STATUS", "RECEIVED_DATE", "DECISION_DATE", "VISA_CLASS",
    "JOB_TITLE", "SOC_CODE", "SOC_TITLE", "FULL_TIME_POSITION",
    "BEGIN_DATE", "END_DATE", "TOTAL_WORKER_POSITIONS",
    "NEW_EMPLOYMENT", "CONTINUED_EMPLOYMENT", "CHANGE_PREVIOUS_EMPLOYMENT",
    "NEW_CONCURRENT_EMPLOYMENT", "CHANGE_EMPLOYER", "AMENDED_PETITION",
    "EMPLOYER_NAME", "EMPLOYER_CITY", "EMPLOYER_STATE", "NAICS_CODE",
    "WORKSITE_CITY", "WORKSITE_STATE",
    "WAGE_RATE_OF_PAY_FROM", "WAGE_RATE_OF_PAY_TO", "WAGE_UNIT_OF_PAY",
    "PREVAILING_WAGE", "PW_UNIT_OF_PAY", "PW_WAGE_LEVEL",
```

```
    "H_1B_DEPENDENT", "WILLFUL_VIOLATOR",
]

UNIT_TO_YEAR = {"YEAR": 1, "MONTH": 12, "BI-WEEKLY": 26, "WEEK": 52, "HOUR": 2080}

# Legal-form suffixes removed so "STRIPE, INC." and "Stripe Inc" become one employer.
SUFFIX = re.compile(
    r"\b(INCORPORATED|INC|LLC|L L C|LLP|LP|LTD|LIMITED|CORPORATION|CORP|CO|COMPANY|PLLC|PC|PBC|"
    r"HOLDINGS|GROUP|USA|US|AMERICA|NA)\b"
)

STATE_ABBR = {
    "ALABAMA": "AL", "ALASKA": "AK", "ARIZONA": "AZ", "ARKANSAS": "AR", "CALIFORNIA": "CA",
    "COLORADO": "CO", "CONNECTICUT": "CT", "DELAWARE": "DE", "DISTRICT OF COLUMBIA": "DC",
    "FLORIDA": "FL", "GEORGIA": "GA", "HAWAII": "HI", "IDAHO": "ID", "ILLINOIS": "IL",
    "INDIANA": "IN", "IOWA": "IA", "KANSAS": "KS", "KENTUCKY": "KY", "LOUISIANA": "LA",
    "MAINE": "ME", "MARYLAND": "MD", "MASSACHUSETTS": "MA", "MICHIGAN": "MI", "MINNESOTA": "MN",
    "MISSISSIPPI": "MS", "MISSOURI": "MO", "MONTANA": "MT", "NEBRASKA": "NE", "NEVADA": "NV",
    "NEW HAMPSHIRE": "NH", "NEW JERSEY": "NJ", "NEW MEXICO": "NM", "NEW YORK": "NY",
    "NORTH CAROLINA": "NC", "NORTH DAKOTA": "ND", "OHIO": "OH", "OKLAHOMA": "OK", "OREGON": "OR",
    "PENNSYLVANIA": "PA", "RHODE ISLAND": "RI", "SOUTH CAROLINA": "SC", "SOUTH DAKOTA": "SD",
    "TENNESSEE": "TN", "TEXAS": "TX", "UTAH": "UT", "VERMONT": "VT", "VIRGINIA": "VA",
    "WASHINGTON": "WA", "WEST VIRGINIA": "WV", "WISCONSIN": "WI", "WYOMING": "WY",
    "PUERTO RICO": "PR", "GUAM": "GU", "VIRGIN ISLANDS": "VI",
}


def normalize_employer(name: object) -> str:
    """Upper-case, strip punctuation and legal suffixes. Pure and unit-tested."""
    if not isinstance(name, str):
        return ""
    s = name.upper()
    # "Formagrid, Inc. (d/b/a Airtable)" -> legal name only, so both spellings merge.
    s = re.sub(r"[\(,]?\s*\bD\s*/?\s*B\s*/?\s*A\b.*$", "", s)
    s = s.replace("&", " AND ")
    s = re.sub(r"[^A-Z0-9 ]", " ", s)
    s = SUFFIX.sub(" ", s)
    s = re.sub(r"\s+", " ", s).strip()
    return s


def normalize_soc(code: object) -> str:
    """'15-1252.00' / '151252' -> '15-1252'. Empty string when unusable."""
    if not isinstance(code, str) and not isinstance(code, (int, float)):
        return ""
    digits = re.sub(r"\D", "", str(code))
    if len(digits) < 6:
        return ""
    return f"{digits[:2]}-{digits[2:6]}"


def normalize_state(s: object) -> str:
    if not isinstance(s, str):
        return ""
    s = s.strip().upper()
    if len(s) == 2:
        return s
    return STATE_ABBR.get(s, "")
```

```
def annualize(amount: object, unit: object) -> float:
    try:
        a = float(str(amount).replace("$", "").replace(",", ""))
    except (TypeError, ValueError):
        return np.nan
    mult = UNIT_TO_YEAR.get(str(unit).strip().upper())
    if mult is None or not np.isfinite(a):
        return np.nan
    return a * mult


def fiscal_quarter(d: pd.Series) -> pd.Series:
    """US federal fiscal year starts 1 Oct. Returns e.g. 'FY2026Q1' and a sortable int."""
    fy = d.dt.year + (d.dt.month >= 10).astype(int)
    q = ((d.dt.month - 10) % 12) // 3 + 1
    return "FY" + fy.astype("Int64").astype(str) + "Q" + q.astype("Int64").astype(str)


def yes(x: pd.Series) -> pd.Series:
    return x.astype(str).str.strip().str.upper().isin(["Y", "YES", "TRUE", "1"])


# ---------------------------------------------------------------- extract
def extract() -> pd.DataFrame:
    files = sorted(glob.glob(os.path.join(RAW, "*.xlsx")) + glob.glob(os.path.join(RAW, "*.csv")))
    if not files:
        sys.exit(f"No LCA files in {RAW}")
    frames = []
    for f in files:
        print(f"extract  {os.path.basename(f)}", flush=True)
        if f.endswith(".csv"):
            df = pd.read_csv(f, dtype=str, low_memory=False)
        else:
            # Read only the needed columns: the full workbook does not fit in 8 GB as strings.
            wanted = set(COLS)
            df = pd.read_excel(f, dtype=str, engine="calamine",
                               usecols=lambda c: str(c).strip().upper() in wanted)
        df.columns = [c.strip().upper() for c in df.columns]
        keep = [c for c in COLS if c in df.columns]
        missing = sorted(set(COLS) - set(keep))
        if missing:
            print(f"         missing columns tolerated: {missing}")
        df = df[keep].copy()
        df["SOURCE_FILE"] = os.path.basename(f)
        frames.append(df)
    raw = pd.concat(frames, ignore_index=True)
    for c in COLS:
        if c not in raw.columns:
            raw[c] = np.nan
    return raw


# ---------------------------------------------------------------- transform
def transform(raw: pd.DataFrame) -> tuple[pd.DataFrame, dict]:
    log: dict = {"rows_raw": len(raw)}
    df = raw.drop_duplicates("CASE_NUMBER", keep="last").copy()
```

```
    log["rows_after_dedup"] = len(df)

    df = df[df["VISA_CLASS"].astype(str).str.upper().str.strip() == "H-1B"]
    log["rows_h1b"] = len(df)

    for c in ["RECEIVED_DATE", "DECISION_DATE", "BEGIN_DATE", "END_DATE"]:
        df[c] = pd.to_datetime(df[c], errors="coerce")
    # DOL partitions the disclosure files by DECISION date, so quarters are assigned by it.
    df = df[df["DECISION_DATE"].notna()]
    log["rows_with_decision_date"] = len(df)

    df["employer_key"] = df["EMPLOYER_NAME"].map(normalize_employer)
    df = df[df["employer_key"] != ""]
    df["soc_code"] = df["SOC_CODE"].map(normalize_soc)
    df["worksite_state"] = df["WORKSITE_STATE"].map(normalize_state)
    df["employer_state"] = df["EMPLOYER_STATE"].map(normalize_state)

    df["wage_annual"] = [annualize(a, u) for a, u in zip(df["WAGE_RATE_OF_PAY_FROM"], df["WAGE_UNIT_OF_PAY"])]
    df["pw_annual"] = [annualize(a, u) for a, u in zip(df["PREVAILING_WAGE"], df["PW_UNIT_OF_PAY"])]
    # Implausible wages (data-entry errors) are nulled, not dropped, so the filing still counts.
    bad = (df["wage_annual"] < 20_000) | (df["wage_annual"] > 1_000_000)
    log["wages_nulled_out_of_range"] = int(bad.sum())
    df.loc[bad, "wage_annual"] = np.nan
    df.loc[(df["pw_annual"] < 15_000) | (df["pw_annual"] > 1_000_000), "pw_annual"] = np.nan

    for c in ["TOTAL_WORKER_POSITIONS", "NEW_EMPLOYMENT", "CONTINUED_EMPLOYMENT",
              "CHANGE_PREVIOUS_EMPLOYMENT", "NEW_CONCURRENT_EMPLOYMENT", "CHANGE_EMPLOYER",
              "AMENDED_PETITION"]:
        df[c] = pd.to_numeric(df[c], errors="coerce").fillna(0).clip(lower=0)

    status = df["CASE_STATUS"].astype(str).str.upper()
    df["certified"] = status.str.startswith("CERTIFIED") & ~status.str.contains("WITHDRAWN")
    df["withdrawn"] = status.str.contains("WITHDRAWN")
    df["denied"] = status.str.contains("DENIED")
    # "New employment" positions are what an OPT -> H-1B hire uses.
    df["new_hire_positions"] = df["NEW_EMPLOYMENT"] + df["CHANGE_EMPLOYER"]
    df["full_time"] = yes(df["FULL_TIME_POSITION"])
    df["h1b_dependent"] = yes(df["H_1B_DEPENDENT"])
    df["willful_violator"] = yes(df["WILLFUL_VIOLATOR"])
    df["pw_level"] = df["PW_WAGE_LEVEL"].astype(str).str.strip().str.upper().replace({"NAN": "", "NONE": ""})
    df["naics2"] = df["NAICS_CODE"].astype(str).str.extract(r"^(\d{2})")[0].fillna("")
    df["fiscal_quarter"] = fiscal_quarter(df["DECISION_DATE"])
    df["job_title"] = df["JOB_TITLE"].astype(str).str.strip()

    log["rows_clean"] = len(df)
    log["employers"] = int(df["employer_key"].nunique())
    log["quarters"] = sorted(df["fiscal_quarter"].unique().tolist())
    return df, log


# ---------------------------------------------------------------- load (star schema)
def load(df: pd.DataFrame) -> dict:
    os.makedirs(STAR, exist_ok=True)

    # dim_employer: one row per standardised employer; display name = most common raw spelling
    emp = (df.groupby("employer_key")
             .agg(employer_name=("EMPLOYER_NAME", lambda s: s.mode().iat[0]),
                  hq_state=("employer_state", lambda s: s.mode().iat[0] if len(s.mode()) else ""),
```

```
                naics2=("naics2", lambda s: s.mode().iat[0] if len(s.mode()) else ""),
                h1b_dependent=("h1b_dependent", "max"),
                willful_violator=("willful_violator", "max"))
            .reset_index())
    emp.insert(0, "employer_id", np.arange(1, len(emp) + 1))

    occ = (df[df["soc_code"] != ""].groupby("soc_code")
            .agg(soc_title=("SOC_TITLE", lambda s: s.mode().iat[0] if len(s.mode()) else ""))
            .reset_index())
    occ["soc_major"] = occ["soc_code"].str[:2]
    occ.insert(0, "occupation_id", np.arange(1, len(occ) + 1))

    loc = (df[["worksite_state"]].drop_duplicates().rename(columns={"worksite_state": "state"})
            .sort_values("state").reset_index(drop=True))
    loc.insert(0, "location_id", np.arange(1, len(loc) + 1))

    dates = pd.to_datetime(df["DECISION_DATE"].dt.normalize().unique())
    dd = pd.DataFrame({"date": dates}).sort_values("date").reset_index(drop=True)
    dd["date_id"] = dd["date"].dt.strftime("%Y%m%d").astype(int)
    dd["year"] = dd["date"].dt.year
    dd["month"] = dd["date"].dt.month
    dd["fiscal_quarter"] = fiscal_quarter(dd["date"])
    dd = dd[["date_id", "date", "year", "month", "fiscal_quarter"]]

    f = df.merge(emp[["employer_id", "employer_key"]], on="employer_key", how="left")
    f = f.merge(occ[["occupation_id", "soc_code"]], on="soc_code", how="left")
    f = f.merge(loc, left_on="worksite_state", right_on="state", how="left")
    f["date_id"] = f["DECISION_DATE"].dt.strftime("%Y%m%d").astype(int)
    fact = f[["CASE_NUMBER", "employer_id", "occupation_id", "location_id", "date_id",
              "job_title", "pw_level", "full_time", "certified", "withdrawn", "denied",
              "TOTAL_WORKER_POSITIONS", "new_hire_positions", "CONTINUED_EMPLOYMENT",
              "wage_annual", "pw_annual", "RECEIVED_DATE", "BEGIN_DATE"]].rename(columns={
        "CASE_NUMBER": "case_number", "TOTAL_WORKER_POSITIONS": "total_positions",
        "CONTINUED_EMPLOYMENT": "continued_positions", "RECEIVED_DATE": "received_date",
        "BEGIN_DATE": "begin_date"})
    fact["occupation_id"] = fact["occupation_id"].astype("Int64")

    tables = {"dim_employer": emp, "dim_occupation": occ, "dim_location": loc,
              "dim_date": dd, "fact_lca": fact}
    if os.path.exists(DB):
        os.remove(DB)
    con = duckdb.connect(DB)
    for name, t in tables.items():
        t.to_parquet(os.path.join(STAR, f"{name}.parquet"), index=False)
        con.register("t", t)
        con.execute(f"CREATE TABLE {name} AS SELECT * FROM t")
        con.unregister("t")
    con.close()
    return {k: len(v) for k, v in tables.items()}


def main() -> None:
    raw = extract()
    df, log = transform(raw)
    os.makedirs(os.path.join(ROOT, "data"), exist_ok=True)
    df.to_parquet(os.path.join(ROOT, "data", "lca_clean.parquet"), index=False)
    counts = load(df)
    log["star_rows"] = counts
```

```
    import json
    with open(os.path.join(ROOT, "data", "etl_log.json"), "w") as fh:
        json.dump(log, fh, indent=2, default=str)
    print(json.dumps(log, indent=2, default=str))


if __name__ == "__main__":
    main()
```

analytics/models.py *(Models M1–M3 and evaluation)*

```
"""
FocusBoth analytics — three predictive models on the cleaned LCA data.

M1  Sponsorship signal (employer-quarter classification)
    Target : employer files >=1 H-1B LCA with new-employment positions in quarter t+1
    Features: only information available at the end of quarter t
    Models : B0 persistence baseline, B1 frequency baseline, logistic regression, XGBoost
    Split  : temporal. Train on target quarters before the last one; test on the last one.

M2  Job title -> SOC occupation (text classification)
    Lets a live Greenhouse job title be linked to DOL occupation statistics.
    Models : majority-class baseline, exact-title lookup baseline, TF-IDF + logistic regression
    Split  : temporal (last quarter held out)

M3  Sponsored wage (regression on log annual wage)
    Features a student actually knows: occupation, worksite state, wage level, full-time, sector
    Models : median-by-SOC baseline, ridge regression, XGBoost
    Split  : temporal (last quarter held out)

Run: python analytics/models.py     (after etl.py)
"""
from __future__ import annotations

import json
import os
import pickle

import numpy as np
import pandas as pd
from scipy import sparse
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression, Ridge
from sklearn.metrics import (average_precision_score, brier_score_loss, f1_score,
                             mean_absolute_error, r2_score, roc_auc_score)
from sklearn.preprocessing import OneHotEncoder, StandardScaler
import xgboost as xgb

ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
DATA = os.path.join(ROOT, "data")
OUT = os.path.join(ROOT, "outputs")
SEED = 42


def qindex(q: pd.Series) -> pd.Series:
    """'FY2026Q1' -> 2026*4 + 0, so consecutive quarters differ by 1."""
    return q.str[2:6].astype(int) * 4 + q.str[-1].astype(int) - 1
```

```
def precision_at_k(y: np.ndarray, p: np.ndarray, k: int) -> float:
    k = min(k, len(y))
    idx = np.argsort(-p)[:k]
    return float(np.mean(y[idx]))


# ============================================================================ M1
def build_panel(df: pd.DataFrame, latest_only: bool = False) -> pd.DataFrame:
    """Employer x quarter panel. Features use quarters <= t only; label is quarter t+1.
    latest_only=True returns unlabeled features at the final quarter (for live scoring)."""
    df = df.copy()
    df["qi"] = qindex(df["fiscal_quarter"])
    df["is_new"] = df["new_hire_positions"] > 0
    df["is_computer"] = df["soc_code"].str.startswith("15-")
    df["wage_ratio"] = df["wage_annual"] / df["pw_annual"]
    g = df.groupby(["employer_key", "qi"])
    eq = g.agg(
        filings=("CASE_NUMBER", "size"),
        new_filings=("is_new", "sum"),
        positions=("TOTAL_WORKER_POSITIONS", "sum"),
        new_positions=("new_hire_positions", "sum"),
        cert_rate=("certified", "mean"),
        withdraw_rate=("withdrawn", "mean"),
        n_soc=("soc_code", "nunique"),
        n_states=("worksite_state", "nunique"),
        computer_share=("is_computer", "mean"),
        level1_share=("pw_level", lambda s: (s == "I").mean()),
        wage_ratio=("wage_ratio", "median"),
        median_wage=("wage_annual", "median"),
        h1b_dependent=("h1b_dependent", "max"),
        naics2=("naics2", "first"),
    ).reset_index()

    qs = sorted(df["qi"].unique())
    rows = []
    for t in ([qs[-1]] if latest_only else qs[:-1]):
        hist = eq[eq["qi"] <= t]
        cur = eq[eq["qi"] == t].set_index("employer_key")
        agg = hist.groupby("employer_key").agg(
            filings_hist=("filings", "sum"),
            new_filings_hist=("new_filings", "sum"),
            quarters_active=("qi", "nunique"),
            last_q=("qi", "max"),
            cert_rate_hist=("cert_rate", "mean"),
            withdraw_rate_hist=("withdraw_rate", "mean"),
            n_soc_hist=("n_soc", "max"),
            n_states_hist=("n_states", "max"),
            computer_share_hist=("computer_share", "mean"),
            level1_share_hist=("level1_share", "mean"),
            wage_ratio_hist=("wage_ratio", "median"),
            median_wage_hist=("median_wage", "median"),
            # employer attributes taken only from quarters <= t (no look-ahead)
            h1b_dependent=("h1b_dependent", "max"),
            naics2=("naics2", "last"),
        )
        agg["filings_t"] = cur["filings"].reindex(agg.index).fillna(0)
        agg["new_filings_t"] = cur["new_filings"].reindex(agg.index).fillna(0)
```

```
        prev = eq[eq["qi"] == t - 1].set_index("employer_key")
        agg["filings_tm1"] = prev["filings"].reindex(agg.index).fillna(0)
        agg["recency_q"] = t - agg["last_q"]
        agg["history_q"] = len([q for q in qs if q <= t])
        nxt = eq[eq["qi"] == t + 1].set_index("employer_key")
        if not latest_only:
            agg["y"] = (nxt["new_filings"].reindex(agg.index).fillna(0) > 0).astype(int)
        agg["t"] = t
        rows.append(agg.drop(columns=["last_q"]).reset_index())
    panel = pd.concat(rows, ignore_index=True)
    return panel


M1_NUM = ["filings_hist", "new_filings_hist", "quarters_active", "cert_rate_hist",
          "withdraw_rate_hist", "n_soc_hist", "n_states_hist", "computer_share_hist",
          "level1_share_hist", "wage_ratio_hist", "median_wage_hist", "filings_t",
          "new_filings_t", "filings_tm1", "recency_q"]


def m1_features(panel: pd.DataFrame, enc: OneHotEncoder | None = None):
    X = panel[M1_NUM].copy()
    for c in ["filings_hist", "new_filings_hist", "filings_t", "new_filings_t", "filings_tm1",
              "n_soc_hist", "n_states_hist"]:
        X[c] = np.log1p(X[c])
    X["median_wage_hist"] = np.log(X["median_wage_hist"].fillna(X["median_wage_hist"].median()))
    X = X.fillna(X.median(numeric_only=True)).fillna(0)
    X["h1b_dependent"] = panel["h1b_dependent"].astype(int).values
    cat = panel[["naics2"]].fillna("")
    if enc is None:
        enc = OneHotEncoder(handle_unknown="ignore", min_frequency=50)
        enc.fit(cat)
    C = enc.transform(cat)
    return X, C, enc


def run_m1(df: pd.DataFrame) -> dict:
    panel = build_panel(df)
    t_test = panel["t"].max()
    tr, te = panel[panel["t"] < t_test], panel[panel["t"] == t_test]
    if len(tr) == 0:
        raise SystemExit("M1 needs at least 3 quarters of data (2 labelled quarters).")
    ytr, yte = tr["y"].values, te["y"].values

    Xtr, Ctr, enc = m1_features(tr)
    Xte, Cte, _ = m1_features(te, enc)
    scaler = StandardScaler().fit(Xtr)
    Atr = sparse.hstack([sparse.csr_matrix(scaler.transform(Xtr)), Ctr]).tocsr()
    Ate = sparse.hstack([sparse.csr_matrix(scaler.transform(Xte)), Cte]).tocsr()

    preds = {
        "B0 Persistence (filed new-hire LCA this quarter)": (te["new_filings_t"].values > 0).astype(float),
        "B1 Frequency (historical new-hire filings)": te["new_filings_hist"].values.astype(float),
    }
    lr = LogisticRegression(max_iter=2000, C=1.0, class_weight="balanced", random_state=SEED)
    lr.fit(Atr, ytr)
    preds["Logistic regression"] = lr.predict_proba(Ate)[:, 1]

    Gtr = sparse.hstack([sparse.csr_matrix(Xtr.values), Ctr]).tocsr()
```

```
    Gte = sparse.hstack([sparse.csr_matrix(Xte.values), Cte]).tocsr()
    gb = xgb.XGBClassifier(n_estimators=400, max_depth=5, learning_rate=0.05, subsample=0.8,
                           colsample_bytree=0.8, eval_metric="logloss", random_state=SEED, n_jobs=2)
    gb.fit(Gtr, ytr)
    preds["XGBoost"] = gb.predict_proba(Gte)[:, 1]

    res = []
    k = max(1, int(round(0.01 * len(yte))))
    for name, p in preds.items():
        r = {"model": name,
             "roc_auc": roc_auc_score(yte, p),
             "pr_auc": average_precision_score(yte, p),
             f"precision_at_top1pct(k={k})": precision_at_k(yte, p, k)}
        if name in ("Logistic regression", "XGBoost"):
            r["brier"] = brier_score_loss(yte, p)
        res.append(r)

    feat_names = list(Xtr.columns) + [f"naics2={c}" for c in enc.get_feature_names_out()]
    imp = pd.Series(gb.feature_importances_, index=feat_names).sort_values(ascending=False)
    coef = pd.Series(lr.coef_[0][: len(Xtr.columns)], index=Xtr.columns).sort_values()

    # Score every employer on the latest quarter with the model refit on all labelled rows.
    full = panel
    Xf, Cf, enc_f = m1_features(full)
    gb_f = xgb.XGBClassifier(**gb.get_params()).fit(
        sparse.hstack([sparse.csr_matrix(Xf.values), Cf]).tocsr(), full["y"].values)
    latest = build_panel(df, latest_only=True)
    Xl, Cl, _ = m1_features(latest, enc_f)
    latest["p_sponsor_next_q"] = gb_f.predict_proba(
        sparse.hstack([sparse.csr_matrix(Xl.values), Cl]).tocsr())[:, 1]

    return {
        "panel_rows": len(panel), "train_rows": len(tr), "test_rows": len(te),
        "test_quarter_index": int(t_test), "positive_rate_train": float(ytr.mean()),
        "positive_rate_test": float(yte.mean()), "results": res,
        "xgb_importance": imp.head(15).to_dict(), "lr_coef": coef.to_dict(),
        "_scores": latest, "_test": te.assign(p_xgb=preds["XGBoost"], p_lr=preds["Logistic regression"]),
    }


# ============================================================================ M2
def clean_title(s: pd.Series) -> pd.Series:
    return (s.astype(str).str.lower().str.replace(r"[^a-z0-9 ]", " ", regex=True)
             .str.replace(r"\s+", " ", regex=True).str.strip())


def run_m2(df: pd.DataFrame, top_n: int = 60) -> dict:
    d = df[(df["soc_code"] != "") & (df["job_title"].str.len() > 1)].copy()
    d["title"] = clean_title(d["job_title"])
    d["qi"] = qindex(d["fiscal_quarter"])
    top = d["soc_code"].value_counts().head(top_n).index
    coverage = float(d["soc_code"].isin(top).mean())
    d = d[d["soc_code"].isin(top)]
    t_test = d["qi"].max()
    tr, te = d[d["qi"] < t_test], d[d["qi"] == t_test]

    maj = tr["soc_code"].mode().iat[0]
    p_maj = np.full(len(te), maj)
```

```
    lookup = tr.groupby("title")["soc_code"].agg(lambda s: s.mode().iat[0])
    p_look = te["title"].map(lookup).fillna(maj).values
    seen = float(te["title"].isin(lookup.index).mean())

    vec = TfidfVectorizer(ngram_range=(1, 2), min_df=2, max_features=20000, sublinear_tf=True)
    Xtr = vec.fit_transform(tr["title"])
    Xte = vec.transform(te["title"])
    clf = LogisticRegression(max_iter=3000, C=5.0, random_state=SEED)
    clf.fit(Xtr, tr["soc_code"])
    proba = clf.predict_proba(Xte)
    p_lr = clf.classes_[proba.argmax(1)]
    top3 = clf.classes_[np.argsort(-proba, 1)[:, :3]]
    y = te["soc_code"].values

    def row(name, p, t3=None):
        r = {"model": name, "top1_accuracy": float(np.mean(p == y)),
             "macro_f1": float(f1_score(y, p, average="macro", zero_division=0)),
             "weighted_f1": float(f1_score(y, p, average="weighted", zero_division=0))}
        r["top3_accuracy"] = float(np.mean([yy in tt for yy, tt in zip(y, t3)])) if t3 is not None else None
        return r

    unseen = ~te["title"].isin(lookup.index).values
    res = [row("Majority class", p_maj), row("Exact-title lookup", p_look),
           row("TF-IDF + logistic regression", p_lr, top3)]
    res_unseen = {"Exact-title lookup": float(np.mean(p_look[unseen] == y[unseen])) if unseen.any() else None,
                  "TF-IDF + logistic regression": float(np.mean(p_lr[unseen] == y[unseen])) if unseen.any()
else None}

    soc_titles = (df[df["soc_code"].isin(top)].groupby("soc_code")["SOC_TITLE"]
                    .agg(lambda s: s.mode().iat[0]).to_dict())
    return {"n_classes": int(len(top)), "row_coverage_of_top_classes": coverage,
            "train_rows": len(tr), "test_rows": len(te), "test_titles_seen_in_train": seen,
            "results": res, "accuracy_on_unseen_titles": res_unseen,
            "_vec": vec, "_clf": clf, "_soc_titles": soc_titles}


# ============================================================================ M3
M3_CAT = ["soc_code", "worksite_state", "pw_level", "naics2", "full_time"]


def run_m3(df: pd.DataFrame) -> dict:
    d = df[df["certified"] & df["wage_annual"].notna() & (df["soc_code"] != "")
           & (df["worksite_state"] != "")].copy()
    d["full_time"] = d["full_time"].astype(str)
    d["qi"] = qindex(d["fiscal_quarter"])
    d["y"] = np.log(d["wage_annual"])
    t_test = d["qi"].max()
    tr, te = d[d["qi"] < t_test], d[d["qi"] == t_test]

    med = tr.groupby("soc_code")["y"].median()
    p0 = te["soc_code"].map(med).fillna(tr["y"].median()).values

    enc = OneHotEncoder(handle_unknown="ignore", min_frequency=20)
    Xtr = enc.fit_transform(tr[M3_CAT])
    Xte = enc.transform(te[M3_CAT])
    ridge = Ridge(alpha=1.0).fit(Xtr, tr["y"])
    p1 = ridge.predict(Xte)
    gb = xgb.XGBRegressor(n_estimators=600, max_depth=7, learning_rate=0.08, subsample=0.8,
```

```
                        colsample_bytree=0.8, random_state=SEED, n_jobs=2)
    gb.fit(Xtr, tr["y"])
    p2 = gb.predict(Xte)

    ytrue = np.exp(te["y"].values)
    res = []
    for name, p in [("Median wage by SOC", p0), ("Ridge regression", p1), ("XGBoost", p2)]:
        pe = np.exp(p)
        res.append({"model": name, "mae_usd": float(mean_absolute_error(ytrue, pe)),
                    "mape": float(np.mean(np.abs(pe - ytrue) / ytrue)),
                    "r2_log": float(r2_score(te["y"], p))})
    names = enc.get_feature_names_out(M3_CAT)
    gimp = pd.Series(gb.feature_importances_, index=names)
    group_imp = gimp.groupby(lambda n: n.split("_")[0] if not n.startswith(("soc_code", "worksite_state",
"pw_level", "full_time"))
                             else ("soc_code" if n.startswith("soc_code") else
                                   "worksite_state" if n.startswith("worksite_state") else
                                   "pw_level" if n.startswith("pw_level") else "full_time")).sum()
    return {"train_rows": len(tr), "test_rows": len(te), "results": res,
            "importance_by_feature": group_imp.sort_values(ascending=False).to_dict(),
            "_test": te.assign(pred_xgb=np.exp(p2))}


# ============================================================================ main
def main() -> None:
    os.makedirs(OUT, exist_ok=True)
    df = pd.read_parquet(os.path.join(DATA, "lca_clean.parquet"))
    print(f"rows={len(df):,} quarters={sorted(df['fiscal_quarter'].unique())}", flush=True)

    m1 = run_m1(df); print("M1 done", flush=True)
    m2 = run_m2(df); print("M2 done", flush=True)
    m3 = run_m3(df); print("M3 done", flush=True)

    m1["_scores"].to_parquet(os.path.join(OUT, "employer_scores.parquet"), index=False)
    m1["_test"].to_parquet(os.path.join(OUT, "m1_test_predictions.parquet"), index=False)
    m3["_test"][["soc_code", "worksite_state", "pw_level", "wage_annual", "pred_xgb"]].to_parquet(
        os.path.join(OUT, "m3_test_predictions.parquet"), index=False)
    with open(os.path.join(OUT, "m2_title_model.pkl"), "wb") as fh:
        pickle.dump({"vec": m2["_vec"], "clf": m2["_clf"], "soc_titles": m2["_soc_titles"]}, fh)

    metrics = {k: {kk: vv for kk, vv in v.items() if not kk.startswith("_")}
               for k, v in {"M1_sponsorship": m1, "M2_title_to_soc": m2, "M3_wage": m3}.items()}
    with open(os.path.join(OUT, "metrics.json"), "w") as fh:
        json.dump(metrics, fh, indent=2, default=float)
    print(json.dumps(metrics, indent=2, default=float))


if __name__ == "__main__":
    main()
```

analytics/robustness.py *(Second fold and bootstrap intervals)*

```
"""
Robustness checks for the reported model comparisons.

1. M1 second temporal fold: train on label quarter FY2026Q1 only, test on FY2026Q2
   (a non-cap-season quarter), so the result does not rest on a single test quarter.
2. Bootstrap 95% confidence intervals (1,000 resamples) on the main M1 test fold for
```

```
   ROC-AUC / PR-AUC of every model, and for the AUC gain of XGBoost over the best baseline.
3. Bootstrap CI for M3 XGBoost MAE; normal-approximation CI for M2 top-1 accuracy.

Run: python analytics/robustness.py   (after models.py)
"""
from __future__ import annotations

import json
import os
import sys

import numpy as np
import pandas as pd
from scipy import sparse
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import average_precision_score, roc_auc_score
from sklearn.preprocessing import StandardScaler
import xgboost as xgb

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from models import SEED, build_panel, m1_features  # noqa: E402

ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
OUT = os.path.join(ROOT, "outputs")
B = 1000
rng = np.random.default_rng(SEED)


def boot(y, scores: dict, fn) -> dict:
    n = len(y)
    stats = {k: [] for k in scores}
    for _ in range(B):
        i = rng.integers(0, n, n)
        if y[i].min() == y[i].max():
            continue
        for k, p in scores.items():
            stats[k].append(fn(y[i], p[i]))
    return {k: [float(np.percentile(v, 2.5)), float(np.percentile(v, 97.5))] for k, v in stats.items()}


def main():
    out = {}
    df = pd.read_parquet(os.path.join(ROOT, "data", "lca_clean.parquet"))
    panel = build_panel(df)
    ts = sorted(panel["t"].unique())

    # ---- 1. second fold
    tr, te = panel[panel["t"] == ts[0]], panel[panel["t"] == ts[1]]
    Xtr, Ctr, enc = m1_features(tr)
    Xte, Cte, _ = m1_features(te, enc)
    sc = StandardScaler().fit(Xtr)
    lr = LogisticRegression(max_iter=2000, class_weight="balanced", random_state=SEED).fit(
        sparse.hstack([sparse.csr_matrix(sc.transform(Xtr)), Ctr]).tocsr(), tr["y"])
    gb = xgb.XGBClassifier(n_estimators=400, max_depth=5, learning_rate=0.05, subsample=0.8,
                           colsample_bytree=0.8, eval_metric="logloss", random_state=SEED, n_jobs=2).fit(
        sparse.hstack([sparse.csr_matrix(Xtr.values), Ctr]).tocsr(), tr["y"])
    y2 = te["y"].values
    p2 = {
```

```
        "B0 Persistence": (te["new_filings_t"].values > 0).astype(float),
        "B1 Frequency": te["new_filings_hist"].values.astype(float),
        "Logistic regression": lr.predict_proba(sparse.hstack([sparse.csr_matrix(sc.transform(Xte)),
Cte]).tocsr())[:, 1],
        "XGBoost": gb.predict_proba(sparse.hstack([sparse.csr_matrix(Xte.values), Cte]).tocsr())[:, 1],
    }
    out["M1_fold2"] = {
        "train_label_quarter": "FY2026Q1", "test_label_quarter": "FY2026Q2",
        "train_rows": len(tr), "test_rows": len(te), "positive_rate_test": float(y2.mean()),
        "results": [{"model": k, "roc_auc": float(roc_auc_score(y2, p)), "pr_auc":
float(average_precision_score(y2, p))}
                    for k, p in p2.items()],
    }
    print("fold2 done", flush=True)

    # ---- 2. bootstrap CIs on the main M1 fold
    m1 = pd.read_parquet(os.path.join(OUT, "m1_test_predictions.parquet"))
    y = m1["y"].values
    p = {"B0 Persistence": (m1["new_filings_t"].values > 0).astype(float),
         "B1 Frequency": m1["new_filings_hist"].values.astype(float),
         "Logistic regression": m1["p_lr"].values, "XGBoost": m1["p_xgb"].values}
    out["M1_bootstrap_roc_auc"] = boot(y, p, roc_auc_score)
    out["M1_bootstrap_pr_auc"] = boot(y, p, average_precision_score)
    diffs = []
    n = len(y)
    for _ in range(B):
        i = rng.integers(0, n, n)
        diffs.append(roc_auc_score(y[i], p["XGBoost"][i]) - roc_auc_score(y[i], p["B1 Frequency"][i]))
    out["M1_auc_gain_xgb_vs_frequency_ci"] = [float(np.percentile(diffs, 2.5)), float(np.percentile(diffs,
97.5))]
    print("bootstrap M1 done", flush=True)

    # ---- 3. M3 MAE CI, M2 accuracy CI
    m3 = pd.read_parquet(os.path.join(OUT, "m3_test_predictions.parquet"))
    err = np.abs(m3["pred_xgb"].values - m3["wage_annual"].values)
    maes = [err[rng.integers(0, len(err), len(err))].mean() for _ in range(B)]
    out["M3_xgb_mae_ci"] = [float(np.percentile(maes, 2.5)), float(np.percentile(maes, 97.5))]
    met = json.load(open(os.path.join(OUT, "metrics.json")))
    n2 = met["M2_title_to_soc"]["test_rows"]
    out["M2_top1_ci"] = {}
    for r in met["M2_title_to_soc"]["results"]:
        a = r["top1_accuracy"]
        h = 1.96 * np.sqrt(a * (1 - a) / n2)
        out["M2_top1_ci"][r["model"]] = [a - h, a + h]

    with open(os.path.join(OUT, "robustness.json"), "w") as fh:
        json.dump(out, fh, indent=2)
    print(json.dumps(out, indent=2))


if __name__ == "__main__":
    main()
```

`lib/compass-engine.ts` *(Deterministic rules engine)*

```
import type { Job, PathEvaluation, PathScenario, VisaProfile } from "./types";

const MS_PER_DAY = 86_400_000;
```

```
export function daysBetween(from: string | Date, to: string | Date) {
  const start = new Date(from);
  const end = new Date(to);
  return Math.ceil((end.getTime() - start.getTime()) / MS_PER_DAY);
}

export function daysUntil(date: string) {
  return Math.max(0, daysBetween(new Date(), date));
}

export function unemploymentAllowance(profile: VisaProfile) {
  const cap = profile.visaStatus === "F-1 STEM OPT" ? 150 : 90;
  return { cap, remaining: Math.max(0, cap - profile.unemploymentDaysUsed) };
}

export function jobPassesConstraints(job: Job, profile: VisaProfile) {
  return (
    job.sponsorScore >= 60 &&
    job.resumeFit >= 65 &&
    new Date(job.startDate) <= new Date(profile.eadEndDate)
  );
}

export function evaluatePath(path: PathScenario, profile: VisaProfile): PathEvaluation {
  const startsBeforeEad = new Date(path.startDate) <= new Date(profile.eadEndDate);
  const enoughHours = path.weeklyHours >= 20;
  const checks = [
    {
      label: "Starts before EAD end",
      detail: `${path.startDate} compared with ${profile.eadEndDate}`,
      pass: startsBeforeEad,
    },
    {
      label: "At least 20 hours each week",
      detail: `${path.weeklyHours} planned hours`,
      pass: enoughHours,
    },
    {
      label: "Employer participates in E-Verify",
      detail: path.employerEnrolledEVerify ? "Recorded as enrolled" : "Not recorded",
      pass: path.employerEnrolledEVerify,
    },
    {
      label: "Bona fide employer relationship",
      detail: path.hasEmployerRelationship ? "Employer relationship present" : "No qualifying relationship
entered",
      pass: path.hasEmployerRelationship,
    },
  ];
  const feasible = checks.every((check) => check.pass);
  const failures = checks.filter((check) => !check.pass).map((check) => check.label.toLowerCase());
  return {
    feasible,
    checks,
    recommendation: feasible
      ? `${path.name} clears all four rules entered in this prototype and provides $
{path.runwayMonths.toFixed(1)} months of runway. Verify the assumptions with your DSO before acting.`
```

```
      : `${path.name} is blocked by ${failures.join(" and ")}. Change the underlying assumptions before
comparing this path on career upside.`,
  };
}

export function applyDestination(job: Pick<Job, "applyUrl" | "openingsUrl">) {
  if (job.applyUrl.trim()) {
    return { url: job.applyUrl.trim(), kind: "posting" as const };
  }
  return { url: job.openingsUrl.trim(), kind: "openings" as const };
}

export function coordinatorBrief(profile: VisaProfile, jobs: Job[]) {
  const allowance = unemploymentAllowance(profile);
  const passing = jobs.filter((job) => jobPassesConstraints(job, profile));
  const next = [...passing].sort((a, b) => b.resumeFit + b.sponsorScore - (a.resumeFit + a.sponsorScore))[0];
  if (!next) {
    return `No tracked role currently clears the sponsor, résumé-fit, and EAD-date rules. Protect the remaining
${allowance.remaining} unemployment days by widening the search rather than lowering every threshold at once.`;
  }
  return `${next.company}'s ${next.role} is the strongest current option: sponsor signal ${next.sponsorScore},
résumé fit ${next.resumeFit}, and a start date before the EAD end. ${allowance.remaining} unemployment days
remain; confirm every date with your DSO before making a decision.`;
}
```

lib/sponsorship.ts *(Sponsorship lookup used by the app)*

```
/**
 * Real H-1B sponsorship signal from U.S. DOL OFLC LCA disclosure data.
 * Built offline by analytics/etl.py + analytics/models.py + analytics/export.py.
 *
 * Server-only: the full employer table is large, so routes attach the few records
 * the browser needs instead of shipping the whole file to the client.
 */
import data from "./data/employer-sponsorship.json";

export type EmployerRecord = {
  name: string;
  filings: number;
  newHireFilings: number;
  certRate: number;
  medianWage: number | null;
  pNextQuarter: number | null;
  lastQuarter: string;
};

/** What the UI shows. `score` is the model probability x 100, never a made-up number. */
export type SponsorSignal = {
  employerKey: string;
  legalName: string;
  score: number | null;
  filings: number;
  newHireFilings: number;
  certRate: number;
  medianWage: number | null;
  quarters: string;
};
```

```
type Payload = { source: string; quarters: string[]; employers: Record<string, EmployerRecord> };
const payload = data as Payload;

const SUFFIX =
  /\b(INCORPORATED|INC|LLC|L L C|LLP|LP|LTD|LIMITED|CORPORATION|CORP|CO|COMPANY|PLLC|PC|PBC|HOLDINGS|GROUP|USA|
US|AMERICA|NA)\b/g;

/** Same rules as normalize_employer() in analytics/etl.py. Keep the two in sync. */
export function normalizeEmployer(name: string): string {
  let s = name.toUpperCase();
  s = s.replace(/[(,]?\s*\bD\s*\/?\s*B\s*\/?\s*A\b.*$/, "");
  s = s.replace(/&/g, " AND ");
  s = s.replace(/[^A-Z0-9 ]/g, " ");
  s = s.replace(SUFFIX, " ");
  return s.replace(/\s+/g, " ").trim();
}

export const dataQuarters = `${payload.quarters[0]}–${payload.quarters[payload.quarters.length - 1]}`;

export function signalForKey(employerKey: string): SponsorSignal | null {
  const record = payload.employers[employerKey];
  if (!record) return null;
  return {
    employerKey,
    legalName: record.name,
    score: record.pNextQuarter === null ? null : Math.round(record.pNextQuarter * 100),
    filings: record.filings,
    newHireFilings: record.newHireFilings,
    certRate: record.certRate,
    medianWage: record.medianWage,
    quarters: dataQuarters,
  };
}

/** Look up any employer name a student types. Exact normalised match only: no fuzzy guessing. */
export function signalForName(name: string): SponsorSignal | null {
  const key = normalizeEmployer(name);
  return key ? signalForKey(key) : null;
}
```

lib/title-model.ts *(TypeScript inference for model M2)*

```
/**
 * Job title -> SOC occupation classifier (model M2), trained in analytics/models.py and
 * exported by analytics/export_title_model.py. TF-IDF (1-2 grams, sublinear tf, l2 norm)
 * followed by multinomial logistic regression, re-implemented here so it runs on the server.
 */
import model from "./data/title-model.json";

type TitleModel = {
  heldOutTop1Accuracy: number;
  classes: string[];
  socTitles: Record<string, string>;
  vocab: Record<string, number>;
  idf: number[];
  coef: number[][];
  intercept: number[];
};
```

```
const m = model as TitleModel;

export type SocPrediction = { soc: string; title: string; probability: number };

/** Same cleaning as clean_title() in analytics/models.py. */
export function cleanTitle(raw: string): string {
  return raw.toLowerCase().replace(/[^a-z0-9 ]/g, " ").replace(/\s+/g, " ").trim();
}

function vectorize(title: string): Map<number, number> {
  const tokens = cleanTitle(title).match(/\b\w\w+\b/g) ?? []; // sklearn default token_pattern
  const terms = [...tokens];
  for (let i = 0; i + 1 < tokens.length; i++) terms.push(`${tokens[i]} ${tokens[i + 1]}`);
  const counts = new Map<number, number>();
  for (const t of terms) {
    const idx = m.vocab[t];
    if (idx !== undefined) counts.set(idx, (counts.get(idx) ?? 0) + 1);
  }
  let norm = 0;
  const vec = new Map<number, number>();
  for (const [idx, tf] of counts) {
    const w = (1 + Math.log(tf)) * m.idf[idx];
    vec.set(idx, w);
    norm += w * w;
  }
  norm = Math.sqrt(norm) || 1;
  for (const [idx, w] of vec) vec.set(idx, w / norm);
  return vec;
}

/** Top-k SOC predictions with softmax probabilities. */
export function predictSoc(title: string, k = 3): SocPrediction[] {
  const x = vectorize(title);
  const scores = m.intercept.map((b, c) => {
    let s = b;
    for (const [idx, w] of x) s += m.coef[c][idx] * w;
    return s;
  });
  const max = Math.max(...scores);
  const exps = scores.map((s) => Math.exp(s - max));
  const total = exps.reduce((a, b) => a + b, 0);
  return exps
    .map((e, c) => ({ soc: m.classes[c], title: m.socTitles[m.classes[c]], probability: e / total }))
    .sort((a, b) => b.probability - a.probability)
    .slice(0, k);
}

export const titleModelAccuracy = m.heldOutTop1Accuracy;
```